M Tejasri
240701557

Ex.2 Low-Code Development Platforms Versus the Traditional Coding Approach: A Comparative Look

Objective

To place low-code and traditional application development side by side and study how each one moves through its build stages, the overall style of working, and the technical compromises each approach brings with it.

The Traditional Route

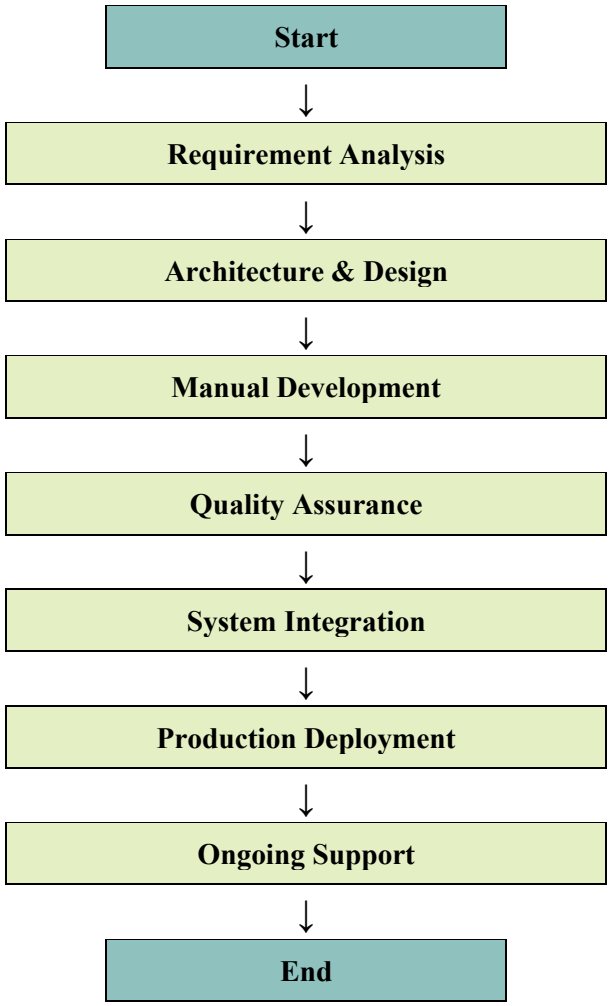
In traditional development, every part of an application's logic is typed out by hand using standard editors, compilers, and integrated development environments. Developers take charge of designing the database structure, building APIs, and setting up deployment infrastructure themselves, which calls for solid command of languages like Java, Python, or C++. This gives full control over the codebase and the system's architecture, but it usually comes with longer build timelines, bigger budgets, and more extensive testing.

The Low-Code Route

Low-code platforms take much of the manual coding burden off developers' shoulders by offering cloud-based, visual building tools instead. Applications are put together using drag-and-drop elements, pre-built templates, and automated deployment pipelines, which makes it possible for "citizen developers" with little to no programming background to produce working software. Because so much of the routine coding is automated, low-code shortens delivery time and reduces initial spending, though this comes at the cost of deep customization and a degree of reliance on the platform itself.

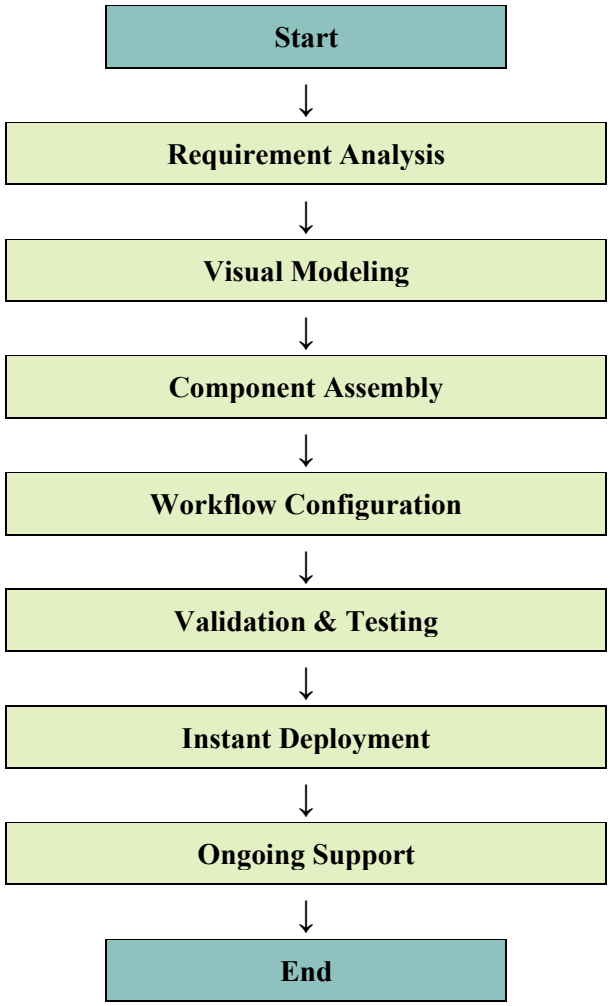
How the Traditional Approach Unfolds

The classic Software Development Life Cycle follows a structured, engineering-driven path, with real effort invested at each and every phase:



How the Low-Code Approach Unfolds

The low-code process replaces hand-coded phases with steps driven by configuration, each one asking for far less direct engineering effort:



Side-by-Side Comparison

Feature	Traditional Development	Low-Code Development
Build Time	Longer, because every module must be individually coded and verified by hand	Much shorter, as prebuilt blocks let teams assemble features quickly
Technical Expertise	Requires developers well-versed in programming languages and frameworks	Needs little to no coding background; visual, drag-and-drop tools make it accessible to citizen developers
Flexibility	Essentially limitless, developers can write any logic the project calls for	Constrained by whatever components and integrations the platform provides
Cost Structure	Higher upfront spend on skilled staff and supporting infrastructure	Cheaper to start, though recurring subscription or licensing charges apply over time
Upkeep	Any change or bug fix means editing code by hand and running it through testing again	The vendor rolls out updates centrally, keeping manual maintenance to a minimum
Dependence on Vendor	None, the company fully owns its source code	Exists, the application stays bound to the provider's platform and ecosystem
Where It Fits Best	Large, performance-demanding, or highly regulated systems	Quick prototypes, internal utilities, and automating routine workflows

Conclusion

This comparison shows that the traditional route still holds the edge for large-scale, high-performance, or heavily regulated systems, where staying in full control is essential. Low-code, in contrast, offers a faster and more budget-friendly path for routine business applications. A growing number of organizations now combine both methods, using low-code for front-end and workflow-related needs while keeping traditional development for the more demanding back-end work.